

University of Pretoria

MIT 805 Group 19

Group Project Part 2

Shreya Bharat
19031786

Lerato Mokori
26834694

TABLE I
DATASET AND SUBSET SIZES

Tier	Size
Raw catalogue	~31 GB
Working set (Part 1)	12.15 GB
Subset (Part 2)	2.58 GiB
Processed by PySpark	4.17 GB

I. INTRODUCTION

The TLC Trip Record Dataset contains data collected by the NYC Taxi and Limousine Commission (TLC) on taxi trip records, including information on trip distances, trip fares, and pickup and drop-off locations. This dataset was chosen to be used for large-scale data processing and analysis, through the use of PySpark.

II. DATASET

Data source: New York City Taxi and Limousine Commission (TLC) Trip Record Data (<https://d37ci6vzurychx.cloudfront.net/trip-data>), used under the NYC.gov Terms of Use, and also mirrored on the AWS Open Data Registry. The dataset does not contain any personally identifiable data. The data is published monthly by NYC TLC.

Table I outlines the raw dataset, and the three data subsets used throughout this study. The working dataset, from part 1, contained 139 months of data, while the subset used in part 2 contained 48 months of data (167,858,646 rows in total). The total size of data processed by PySpark was 4.17 GB, spanning 156,964,574 rows in total.

The subset chosen was data between January 2022 and December 2025. This subset was chosen to exclude data collected during the COVID-19 period, which would have distorted the data towards abnormal operating conditions if it were included. The subset also contains data from January 2025, during which there was a congestion-pricing change, making the data relevant to current policies.

Table II provides a summary of the filtering that was conducted on the raw dataset, in order to derive the data

TABLE II
DATASET FILTERING

Filter	Rows passing	% retained
Initial rows	167,858,646	100.00%
Time-frame (2022–2025)	167,857,980	100.00%
Drop-off > pickup	167,251,228	99.64%
Duration > 0	167,251,228	99.64%
1 min < duration < 3 hr	165,152,624	98.39%
0.1 mi < distance < 100 mi	163,510,043	97.41%
1 mph < speed < 70 mph	163,174,278	97.21%
\$0.01 < fare < \$1000	163,575,252	97.45%
Zone ID between 1 and 263	165,660,093	98.69%
Final subset	156,964,574	93.51%

subset that was used in this part of the project. Filtering was done to ensure that any incorrect data entries were excluded from the analysis, and that only trips in the specified time period were included. Each filter was applied independently to the dataset, resulting in overlap between the filtered rows. After applying the filters together to the raw dataset, a total of 10,894,072 rows were removed.

One of the challenges with the dataset was in handling some of the inaccuracies present, which resulted in the need to filter the raw dataset. For example, it was found that 0.36% of trips had a recorded drop-off at, or before, the pick-up, resulting in zero or negative trip durations.

III. PROCESSING OBJECTIVE AND ANALYTICAL QUESTION

This study aims to answer the following research questions: RQ1: Where does congestion cost the yellow taxi fleet the most revenue? RQ2: Which routes experience the most congestion during peak hours?

Both questions were answered by following the same methodology: a free-flow baseline was first determined, from which the time-loss per trip was calculated. This value was then converted into a monetary value. The same data records were utilised to answer both questions, with only the grouping key differing between the two executions. For RQ1, the data was grouped by (pickup zone, hour, weekday or weekend), and for RQ2 the data was grouped by (pickup zone, drop-off

```

1 pairs = rows.map(lambda r: ((r["pu"], r["hour"], r["daytype"]),
2                             (r["revenue"], r["duration_hr"], 1)))

```

Fig. 1. RDD formulation for mapping

zone, time window).

For the free-flow baseline, off-peak trip data from the same dataset was used to derive the baseline. For each pickup-dropoff pair, the baseline was determined by analysing weekday trips between 01:00 am and 04:59 am by dividing the total distance travelled by the total duration. An assumption was made that trips during this time window will experience the least congestion during the weekdays, and will therefore provide a baseline that peak-traffic trips can be compared to. To ensure an accurate baseline was generated, the study only included routes with more than 100 trips in the baseline. This resulted in 3,599 routes being included in the study. Each route was compared with its own respective baseline, instead of using an overall average, to ensure that results were an accurate representation of each route. If an average baseline was used throughout, the results may have been skewed by high-traffic routes, such as airport routes, which would still experience a higher volume of trips during the off-peak hours compared to routes in quieter residential areas.

Four measures were obtained after the baselines were constructed: 1. Travel Time Index (TTI): free-flow baseline traffic speed divided by the actual speed. 2. Delay hours: actual trip duration minus the duration of the trip if taken at the free-flow speed. 3. Yield: revenue per hour generated. 4. Revenue lost / forgone: the difference between the estimated baseline yield (without congestion) and the actual yield, multiplied by the total hours worked.

The research questions were chosen to outline the overall effect of congestion on revenue generated across the routes travelled. For yellow taxi trips, the meter charges for elapsed time as well as distance. During peak congestion trips, the fare of the individual trip increases, while the number of trips that the driver completes decreases for that time period. The revenue generated per vehicle hour is therefore affected. For this study, revenue was calculated as the sum of the base fare amount and the tip amount, excluding tolls and surcharges.

IV. MAPPING / TRANSFORMATION

Figure 1 shows the RDD formulation that was used for the map function implementation. The aim of the mapping function is to produce the key that is used to answer the research questions.

Each record produced by the map function carries five values, namely distance, duration, revenue, delay hours, and trip count. These are all summed values, and not averages, which allows the reduce function to be able to add the partial

TABLE III
COMPARISON OF REDUCEBY AND GROUPBY OPERATIONS

Operation	Shuffle write	Execution time
reduceByKey	0.207 MB	11.1 s
groupByKey	29.112 MB	14.3 s

results in any order without changing the result. The averages were calculated after the reduction step.

V. SHUFFLE / GROUPING

Rows that share the same key were first grouped together before they could be reduced. This was done by assigning each record to a partition according to a hash of its key. All records which shared a key were assigned to a single partition, and each partition was then processed as a single task. This resulted in an uneven workload in the cases where some keys contained more records than others.

The volume of data crossing the network is largely dependent on whether the reduction is combined with the map function before the shuffle. In order to measure the difference between the reduceBy and groupBy operations, both aggregations were computed over a single month of data. The reduceByKey applied the reduction locally within each partition before the exchange, while the groupByKey did not. The results from this comparison are shown in table III. Both operations produced 10,142 rows of results.

VI. REDUCTION / AGGREGATION

The reduce function must return the same result regardless of the order in which the partial results are combined. In the case of Spark, the partial results were combined concurrently across the tasks, and then once again at the reducer, which did not guarantee a fixed order. In order to work around this challenge, the rates calculated were calculated as sum values, and not as averages. For example, in equation 1, the speed was calculated as the sum of the trip distance, d , divided by the total trip duration, t , for the group. Per-trip averages were not calculated, as using this approach would have resulted in shorter trips holding the same weight as longer trips, whereas computing a ratio of the sums takes into account the distance travelled in the given time-frame and returns a weighted value. The trip yield, defined as the total revenue divided by total trip duration, was calculated similarly.

$$\bar{v}_g = \frac{\sum_{i \in g} d_i}{\sum_{i \in g} t_i} \quad (1)$$

The following values were summed during the shuffle reduction: distance, duration, revenue, delay hours, and trip count. The rates were calculated once, after the completion of the reduction step. The reduction process was verified by first implementing using RDD (using the map and reduceByKey

```

1 == Physical Plan ==
2 InMemoryTableScan (1)
3 +- InMemoryRelation (2)
4   +- Project (46)
5     +- BroadcastHashJoin Inner BuildRight (44)
6       +- Project (39)
7         +- HashAggregate (38)
8           +- Exchange (37)
9             +- HashAggregate (36)
10              +- Project (35)
11                +- BroadcastHashJoin LeftOuter BuildRight (34)
12                  +- Project (21)
13                    +- BroadcastHashJoin LeftOuter BuildRight (20)
14                      +- Project (6)
15                        +- Filter (5)
16                          +- ColumnToRow (4)
17                            +- Scan parquet (3)
18                              +- BroadcastExchange (19)
19                                +- Filter (18)
20                                  +- InMemoryTableScan (7) (columnarIn=false, columnarOut=true)
21                                    +- InMemoryRelation (8)
22                                      +- Project (17)
23                                        +- Filter (16)
24                                          +- HashAggregate (15)
25                                            +- Exchange (14)
26                                              +- HashAggregate (13)
27                                                +- Project (12)
28

```

Fig. 2. Spark execution for RQ1

operations), and then mirroring the process with a DataFrame version (using groupBy and aggregate operations). The results were compared and were found to match across all 12,270 rows computed.

The reduction process was executed to produce individual result sets for RQ1 and RQ2. For RQ1, a total of 12,173 rows were produced, and for RQ2 a total of 14,514 rows were produced, each with over 500 trips.

VII. SPARK EXECUTION AND DAG

For the Spark execution, the transformations were collected into a directed acyclic graph (DAG), which is only executed once an action requires a result. Therefore, the transformations were not executed in the exact order in which they were written.

Figure 2 shows the plan that was executed to obtain the results for RQ1, and figure 3 shows the execution plan for RQ2. The execution plan was obtained using the code block shown in figure 4.

The first part of the Spark execution involved scanning the Parquet files to keep only the required rows and columns, and then aggregating the data within each partition. Exchange hash partitioning was then performed on the grouping key, after which the data was again aggregated, performing the shuffle operation. BroadcastHashJoin was used to add the zone lookup data, since the lookup table was fairly small compared to the primary dataset.

The two aggregation stages were also written against the RDD API, which used the reduceByKey. It was found that the RDD execution ran approximately 42 times slower than the DataFrame execution. One possible reason for this difference in execution times could be that the DataFrame execution only read the required columns from the Parquet files, while the RDD execution built a Python object for every row.

```

1 == Physical Plan ==
2 InMemoryTableScan (1)
3 +- InMemoryRelation (2)
4   +- Project (27)
5     +- Project (26)
6       +- BroadcastHashJoin Inner BuildRight (25)
7         +- Project (11)
8           +- Filter (10)
9             +- HashAggregate (9)
10              +- Exchange (8)
11                +- HashAggregate (7)
12                  +- Project (6)
13                    +- Filter (5)
14                      +- ColumnToRow (4)
15                        +- Scan parquet (3)
16                          +- BroadcastExchange (24)
17                            +- Filter (23)
18                              +- InMemoryTableScan (12) (columnarIn=false, columnarOut=true)
19                                +- InMemoryRelation (13)
20                                  +- Project (22)
21                                    +- Filter (21)
22                                      +- HashAggregate (20)
23                                        +- Exchange (19)
24                                          +- HashAggregate (18)
25                                            +- Project (17)
26                                              +- Filter (16)
27                                                +- ColumnToRow (15)
28                                                  +- Scan parquet (14)

```

Fig. 3. Spark execution for RQ2

```

1 def capture_plan(df, name):
2     buf = io.StringIO()
3     with contextlib.redirect_stdout(buf):
4         df.explain("formatted")
5     text = buf.getvalue()
6     (LOCAL_OUT / f"plan_{name}.txt").write_text(text)
7     print(text[:1800])
8     return text
9
10 _ = capture_plan(rq1, "rq1_zone_hour")
11 _ = capture_plan(rq2, "rq2_route_window")

```

Fig. 4. Spark execution plan

VIII. RESULTS AND TECHNICAL INTERPRETATION

xxx

IX. VISUALISATION

xxx

X. BUSINESS / SOCIETAL INTERPRETATION AND VALUE

xxx

XI. GITHUB REPOSITORY

The project setup and code can be found in the following GitHub repo: <https://github.com/ShreyaB143/MIT805-GroupProject>

TABLE IV
STAGE METRICS, SIGNIFICANT STAGES ONLY

Stage	Role	Tasks	Med. (ms)	Shuffle W	Skew
4	Filtering write	29	11,018	7,766 MB	1.3
5	Filtering read	32	16,250	—	3.7
21	RQ2 aggregation	48	1,165	108.3 MB	3.0
37	RQ1 aggregation	48	1,356	17.7 MB	2.5
54	DataFrame twin	48	1,127	11.6 MB	1.3
56	RDD twin	48	52,202	10.2 MB	1.2

